Chapter 20 | Exercise solutions with Python

Search, Retrieval, and Grounded Assistants

This notebook solves Exercises 5–8. It uses three fictional office notices to follow evidence from lexical search through reranking and passage packing. The last exercise checks how a literal phrase can occur in two passages that make opposite claims.

The examples use the chapter's local NumPy implementations. Each section explains the expected result and checks it against the returned scores, ranks, or passage IDs. The written solutions cover Exercises 1–4 and 9–11.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch20_retrieval"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Turn off length adjustment

P1 tells a resident where to renew a parking permit. P2 gives the expiry date. Both notices contain *parking* and *permit* once, so the two notices are a good test of how BM25 uses passage length. Score the three notices from Table 20.1 in ID order for `parking permit`. Then change only the length parameter `b` from its default value of `0.75` to `0`.

```
from examples.ch20_retrieval.model import bm25_scores, rank_passages

passage_ids = ("P1", "P2", "P3")
passage_texts = (
    "Renew a parking permit at the mobility desk.",
    "A parking permit expires on June 30.",
    "The recycling office opens on Tuesday.",
)
query = "parking permit"

default_scores = bm25_scores(query, passage_texts)
no_length_scores = bm25_scores(query, passage_texts, b=0)

print(f"{'Passage':<8} {'Default':>10} {'b = 0':>10}")
for passage_id, default, no_length in zip(
    passage_ids, default_scores, no_length_scores
):
    print(f"{passage_id:<8} {default:>10.6f} {no_length:>10.6f}")

print("Default order", rank_passages(passage_ids, default_scores))
print("b = 0 order", rank_passages(passage_ids, no_length_scores))
```

```
OUTPUT
Passage     Default      b = 0
P1         0.888105   0.940007
P2         0.940007   0.940007
P3         0.000000   0.000000
Default order ('P2', 'P1', 'P3')
b = 0 order ('P1', 'P2', 'P3')
```

The passages have 8, 7, and 6 word tokens, so the collection average is 7. Each query word occurs in two of the three passages and contributes the same inverse document frequency, $\ln(1.6) \approx 0.470004$. In P1 and P2, each query word occurs once. With the chapter's default $k_1 = 1.2$, the score for either matching passage of length L is therefore

$$2 \ln(1.6) \frac{2.2}{1 + 1.2(1 - b + bL/7)}.$$

With $b = 0.75$, the denominator is about 2.329 for P1 and 2.2 for P2. The smaller denominator gives the shorter notice, P2, the higher score. P3 contains neither query word and scores zero in both runs.

With $b = 0$, both denominators are 2.2 and both scores equal $2 \ln(1.6)$. `rank_passages` orders equal scores by ascending passage ID, so P1 comes first. The rank after the tie break carries no extra evidence about which passage best answers the question.

```
# Check both settings against the hand calculation above.
average_length_score = 2 * np.log(1.6)
long_passage_denominator = 1 + 1.2 * (0.25 + 0.75 * 8 / 7)
long_passage_score = average_length_score * 2.2 / long_passage_denominator
np.testing.assert_allclose(
    default_scores, [long_passage_score, average_length_score, 0]
)
np.testing.assert_allclose(
    no_length_scores, [average_length_score, average_length_score, 0]
)
assert rank_passages(passage_ids, default_scores) == ("P2", "P1", "P3")
assert rank_passages(passage_ids, no_length_scores) == ("P1", "P2", "P3")
assert rank_passages(("P2", "P1"), no_length_scores[[1, 0]]) == (
    "P1", "P2"
)
```

6. Rerank only what was found

For “Where do I renew a parking permit?” the only passage that supplies the location is P1. Compare a retrieval run that returned P2 and P3 with one that also returned P1. Keep P2 and P3’s scores fixed so we can see what changes when P1 becomes available.

```
from examples.ch20_retrieval.model import evidence_recall, rerank_candidates

relevant_ids = ("P1",)
pair_scores = {"P2": 0.2, "P3": 0.7}

candidate_sets = (
    (("P2", "P3"), pair_scores, ("P3", "P2"), 0.0),
    (
        ("P2", "P3", "P1"),
        {**pair_scores, "P1": 0.9},
        ("P1", "P3", "P2"),
        1.0,
    ),
)
for candidates, scores, expected_order, expected_recall in candidate_sets:
    reranked = rerank_candidates(candidates, scores)
    recall = evidence_recall(reranked, relevant_ids, k=1)
    assert reranked == expected_order
    assert recall == expected_recall
    print("Candidates", candidates)
    print("Reranked  ", reranked)
    print("Recall@1  ", recall)
    print()
```

```
OUTPUT
Candidates ('P2', 'P3')
Reranked   ('P3', 'P2')
Recall@1   0.0

Candidates ('P2', 'P3', 'P1')
Reranked   ('P1', 'P3', 'P2')
Recall@1   1.0
```

Recall at rank one is the number of required evidence IDs in the first result divided by the total number of required IDs. Here there is one required ID, P1.

- With **P2 and P3**, P3 comes first because $0.7 > 0.2$. The top result contains zero of the one required IDs, giving recall $0/1 = 0$.
- With **P2, P3, and P1**, the order is P1, P3, P2 because $0.9 > 0.7 > 0.2$. The top result contains P1, giving recall $1/1 = 1$.

Retrieval chooses which passages the reranker can consider. Reranking changes their order using the supplied query–passage scores. When retrieval includes P1, its score moves it from third to first. When retrieval omits P1, changing the order of P2 and P3 leaves the evidence absent.

7. Keep the passage and ID together

The answer to “Where can I renew a parking permit?” is in P1. Begin with the ranking P2, P1, P3 and see whether P1 makes it into the context. Then put P1 first and repeat the comparison at the same budget of 14 tokens.

A passage’s cost is its number of tokens under the chapter’s teaching tokenizer. It lowercases words, ignores the final periods, and counts the number `30` as one token. The budget here covers only passage text. The ID travels with the text as a separate field.

```
from examples.ch20_retrieval.model import (
    literal_evidence_hits,
    pack_context,
    tokenize,
)

packing_corpus = {
    "P1": "Renew a parking permit at the mobility desk.",
    "P2": "A parking permit expires on June 30.",
    "P3": "The recycling office opens on Tuesday.",
}
passage_costs = {
    passage_id: len(tokenize(passage))
    for passage_id, passage in packing_corpus.items()
}
assert passage_costs == {"P1": 8, "P2": 7, "P3": 6}

for passage_id, passage in packing_corpus.items():
    words = " ".join(tokenize(passage))
    print(f"{passage_id} ({passage_costs[passage_id]} tokens)  {words}")
```

```
OUTPUT
P1 (8 tokens)  renew a parking permit at the mobility desk
P2 (7 tokens)  a parking permit expires on june 30
P3 (6 tokens)  the recycling office opens on tuesday
```

`pack_context` reads the ranking from left to right. It keeps a whole passage if its cost is at most the remaining budget. If a passage is too long, it skips that passage and tries the next one. Each retained pair includes the original passage ID and the full text from the dictionary.

```
original_order = ("P2", "P1", "P3")
p1_first_order = ("P1", "P2", "P3")
packing_cases = [
    (original_order, 7, ("P2",), 0, ()),
    (original_order, 14, ("P2", "P3"), 1, ()),
    (p1_first_order, 14, ("P1", "P3"), 0, ("P1",)),
]

for order, budget, expected_ids, left, expected_hits in packing_cases:
    context = pack_context(order, packing_corpus, budget)
    hits = literal_evidence_hits(context, "mobility desk")
    used = sum(passage_costs[passage_id] for passage_id, _ in context)

    expected_context = tuple(
        (passage_id, packing_corpus[passage_id])
        for passage_id in expected_ids
    )
    assert context == expected_context
    assert budget - used == left
    assert hits == expected_hits

    print(f"{' → '.join(order)} with budget {budget}")
    for passage_id, passage in context:
        print(f"  {passage_id}  {passage}")
    print(f"  Tokens left {budget - used}. Phrase hits {hits}.")
```

```
OUTPUT
P2 → P1 → P3 with budget 7
  P2  A parking permit expires on June 30.
  Tokens left 0. Phrase hits ().
P2 → P1 → P3 with budget 14
  P2  A parking permit expires on June 30.
  P3  The recycling office opens on Tuesday.
  Tokens left 1. Phrase hits ().
P1 → P2 → P3 with budget 14
  P1  Renew a parking permit at the mobility desk.
  P3  The recycling office opens on Tuesday.
  Tokens left 0. Phrase hits ('P1',).
```

- **P2, P1, P3 with budget 7.** P2 uses all seven tokens. Neither P1 nor P3 fits in the zero tokens left. The packed context contains only P2, and the phrase has no hits.
- **P2, P1, P3 with budget 14.** P2 leaves seven tokens. P1 needs eight, so it is skipped. P3 needs six, so it fits and leaves one token. The packed context contains P2 and P3, and the phrase has no hits.

- **P1, P2, P3 with budget 14.** P1 leaves six tokens. P2 needs seven, so it is skipped. P3 fits exactly. The packed context contains P1 and P3, and the hit is P1.

The same three passages were retrieved in every run. At budget 14, the first of P1 and P2 to be visited uses enough of the budget to exclude the other, since their combined cost is 15. The first two runs lose the renewal location during packing. Keeping each ID attached to its text lets us check which passage actually reached the input and what a citation to that passage would support.

8. Probe a literal match (optional)

The claim we want to check is "Express renewals are available." Search the two notices for the shorter phrase `express renewals`. The function lowercases both the phrase and the passage text, then checks whether the phrase's tokens occur together in order within each passage. The rest of the sentence does not enter that check.

```
from examples.ch20_retrieval.model import literal_evidence_hits

renewal_context = (
    ("P4", "Express renewals are available"),
    ("P5", "Express renewals are no longer available"),
)
renewal_hits = literal_evidence_hits(
    renewal_context, "express renewals"
)
assert renewal_hits == ("P4", "P5")

for passage_id, passage in renewal_context:
    print(f"{passage_id}  {passage}")
print(f"Phrase hits {renewal_hits}")
```

```
OUTPUT
P4  Express renewals are available
P5  Express renewals are no longer available
Phrase hits ('P4', 'P5')
```

Both notices begin with the consecutive tokens `express` and `renewals`, so both IDs match. P4 states that express renewals are available. In P5, "no longer" changes the claim about current availability. P5 contradicts the positive claim.

A phrase hit can help locate the passage to read. To judge support for the positive claim, read what each passage says about the service, including any negation. A notice that says "You can renew through the express service" could support the same claim even though this phrase search would miss it.